NATIONAL RESEARCH UNIVERSITY OF INFORMATION TECHNOLOGY, MECHANICS AND OPTICS

FACULTY OF CONTROL SYSTEMS AND ROBOTICS

LABORATORY TASK N°2

COMPLETED BY: **Valverde Ramírez, Edgar David**

ISU: **520193**

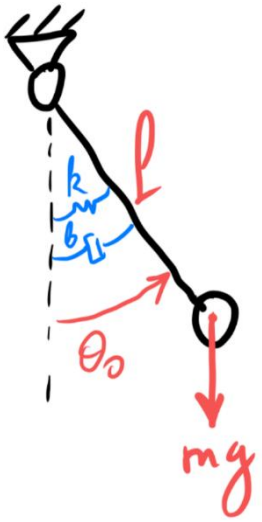
DISCIPLINE: **Simulation of Robotic Systems**

INSTRUCTOR: **PhD. Ivan Borisov**

1. **Data:**
   El objetivo en esta tarea es analizar el comportamiento de estos 2 sistemas dinámicos governed by a linear second-order ordinary differential equation (ODE) with constant coefficients. The specific system parameters provided for the simulation are listed below:

Parameter	Value
m (kg)	0.5
k (N/m) (Nm/rad)	9.4
b (N s/m) (Nm s/rad)	0.005
l (m)	0.95
$\theta_0\ (rad)$	0.109959158
$x_0\ (m)$	0.45

2. **Derivacion de las ecuaciones:**
   Procedemos a definir cada uno de los 2 sistemas planteados.
   Sistema 1:



   Energia Cinetica (T):
   Partimos de la velocidad tangencial de la masa $v = l\dot{\theta}$

$$T = \frac{1}{2}mv^2 = \frac{1}{2}ml^2\dot{\theta}^2$$

   Energia Potencial (V):
   Hay 2 contribuciones:

- Gravitatoria: tomando como nivel de referencia el punto más bajo de la masa, que se da cuando $\theta = 0$, entonces:

$$V_1 = mgl(1 - \cos\theta)$$

- Elastica:
  Se define como:

$$V_2 = \frac{1}{2}k\theta^2$$

Total:

$$L = T - V = \frac{1}{2}ml^2\dot{\theta}^2 - mgl(1-\cos\theta) - \frac{1}{2}k\theta^2$$

La cuasi fuerza en este sistema es: $Q = -b\dot{\theta}$

Replace this terms on lagrange ecuation:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}}\right) - \frac{\partial L}{\partial \theta} = Q$$

Calculamos cada termino:

$$\frac{\partial L}{\partial \dot{\theta}} = ml^2\dot{\theta}, \qquad \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}}\right) = ml^2\ddot{\theta}$$

$$\frac{\partial L}{\partial \theta} = -mgl\sin\theta - k\theta$$

Sustituimos:

$$ml^2\ddot{\theta} + mgl\sin\theta + k\theta = -b\dot{\theta}$$

Reordenando:

$$ml^2\ddot{\theta} + b\dot{\theta} + mgl\sin\theta + k\theta = 0$$

El sistema es no lineal, renombramos las variables:

$$x_1 = \theta, \qquad x_2 = \dot{\theta}$$

Entonces

$$\dot{x_1} = f_1(x_1, x_2) = x_2$$

$$\dot{x_2} = f_2(x_1, x_2) = -\frac{b}{ml^2}x_2 - \frac{g}{l}\sin x_1 - \frac{k}{ml^2}x_1$$

Linealizamos el sistema, mediante el jacobiano:

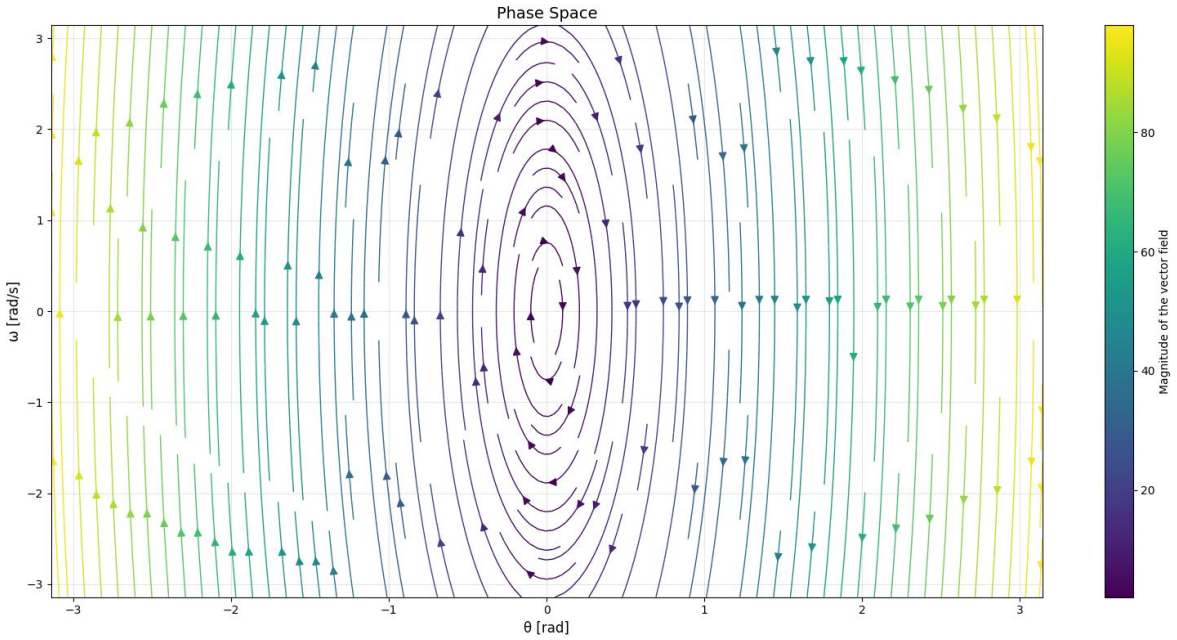
$$J = \begin{bmatrix} f_{1_{x_1}} & f_{1_{x_2}} \\ f_{2_{x_1}} & f_{2_{x_2}} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l}\cos x_1 - \frac{k}{ml^2} & -\frac{b}{ml^2} \end{bmatrix}$$

A partir del jacobiano calculado, consideramos la aproximación para angulos pequeños $x_1 = 0$:

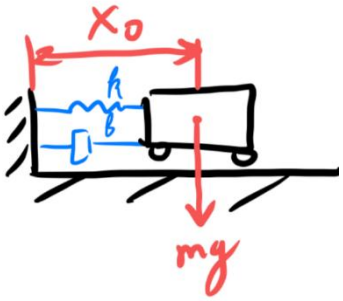
$$\begin{bmatrix} \dot{x_1} \\ \dot{x_2} \end{bmatrix} = J|_{x_1=0}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\begin{bmatrix} \dot{x_1} \\ \dot{x_2} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l} - \frac{k}{ml^2} & -\frac{b}{ml^2} \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Mostramos el espacio de fases del sistema linealizado



Sistema 2:



Energia Cinetica (T):
Partimos de la velocidad del carrito

$$T = \frac{1}{2}m\dot{x}^2$$

Energia Potencial (V):

- Elastica:
  Se define como:

$$V_2 = \frac{1}{2}kx^2$$

Total:

$$L = T - V = \frac{1}{2}m\dot{x}^2 - \frac{1}{2}kx^2$$

La cuasi fuerza en este sistema es: $Q = -b\dot{x}$
Replace this terms on lagrange ecuation:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}}\right) - \frac{\partial L}{\partial \theta} = Q$$

Calculamos cada termino:

$$\frac{\partial L}{\partial \dot{x}} = m\dot{x}, \qquad \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{x}}\right) = m\ddot{x}$$

$$\frac{\partial L}{\partial x} = -kx$$

Sustituimos:

$$m\ddot{x} + kx = -b\dot{x}$$

Reordenando:

$$m\ddot{x} + b\dot{x} + kx = 0$$

El sistema es lineal, renombramos las variables:

$$x_1 = x, \qquad x_2 = \dot{x}$$

Entonces

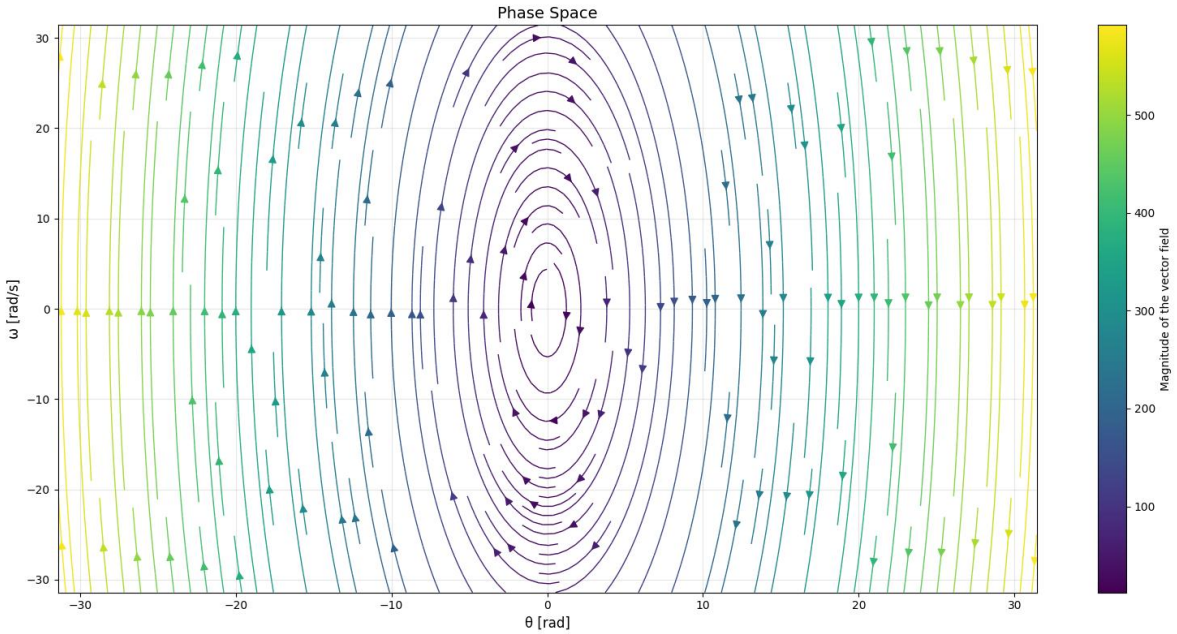
$$\dot{x_1} = f_1(x_1, x_2) = x_2$$

$$\dot{x_2} = f_2(x_1, x_2) = -\frac{b}{m}x_2 - \frac{k}{m}x_1$$

Representando de forma matricial:

$$\begin{bmatrix} \dot{x_1} \\ \dot{x_2} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{b}{m} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Mostramos el espacio de fases del sistema:



El objetivo en esta tarea es analizar el comportamiento de estos 2 sistemas dinámicos

3. **Analytical Solution:**

Sistema 1:

Partimos de la versión linealizada para angulos pequeños:

$$ml^2\ddot{\theta} + b\dot{\theta} + (mgl + k)\theta = 0$$

$$\ddot{\theta} + \frac{b}{ml^2}\dot{\theta} + \frac{(mgl + k)}{ml^2}\theta = 0$$

Reemplazamos los datos:

$$\ddot{\theta} + 0.01108\dot{\theta} + 31.15734\theta = 0$$

Calculamos las raíces de la ecuación característica:

$$r^2 + 0.01108r + 31.15734 = 0$$

$$r = -0.00554 \pm 5.58187i$$

La solución es:

$$\theta(t) = e^{-0.00554t}(C_1 \cos(5.58187t) + C_2 \sin(5.58187t))$$

Consideramos los valores iniciales $\theta_0 = 0.109959158,\ \dot{\theta}_0 = 0$:

$$0.109959158 = \mathrm{C}_1$$

$$0 = -0.00554C_1 + 5.58187C_2$$

$$C_2 = 1.09134 \times 10^{-4}$$

El resultado es:

$$\theta(t) = e^{-0.00554t}(0.109959158 \cos(5.58187t) + 1.09134 \times 10^{-4} \sin(5.58187t))$$

Sistema 2:

$$m\ddot{x} + b\dot{x} + kx = 0$$

$$\ddot{x} + \frac{b}{m}\dot{x} + \frac{k}{m}x = 0$$

Reemplazamos

$$\ddot{x} + 0.01\dot{x} + 18.8x = 0$$

Calculamos las raíces de la ecuación característica

$$r^2 + 0.01r + 18.8 = 0$$

$$r = 0.005 \pm 4.335894i$$

La solución es:

$$x(t) = e^{0.005t}(C_1 \cos(4.335894t) + C_2 \sin(4.335894t))$$

Consideramos las condiciones iniciales $x_0 = 0.45, \dot{x}_0 = 0$

$$0.45 = C_1$$

$$0 = 0.005C_1 + 4.335894C_2$$

$$C_2 = -5.1892 \times 10^{-4}$$

El resultado es:

$$x(t) = e^{0.005t}(0.45 \cos(4.335894t) - 5.1892 \times 10^{-4} \sin(4.335894t))$$

4. **Numerical Solution:**
   To enable numerical simulation, the second-order ODE is transformed into a state-space representation consisting of a system of two first-order ODEs.
   The system can then be expressed in matrix form $\dot{X} = AX$:
   Sistem 1:

$$\begin{bmatrix} \dot{x_1} \\ \dot{x_2} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l} - \frac{k}{ml^2} & -\frac{b}{ml^2} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$
$$F(X) = \dot{X} = AX$$

   Sistem 2:

$$\begin{bmatrix} \dot{x_1} \\ \dot{x_2} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{b}{m} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$
$$F(X) = \dot{X} = AX$$

   a. Explicit Euler
      This first-order method estimates the next state based on the gradient at the current time step. It is computationally simple but conditionally stable. The iteration is given by:

$$X_{i+1} = X_i + hF(X_i)$$
$$X_{i+1} = X_i + h(AX_i)$$

   b. Simplectic Euler
      Este metodo de primer orden Estima de forma explicita la velocidad y la usa para calcular de forma implícita la posición, aplicado en sistemas hamiltonianos:

$$x_{2_{i+1}} = x_{2_i} + hf_2(x_{1_i}, x_{2_i})$$
$$x_{1_{i+1}} = x_{1_i} + hx_{2_{i+1}}$$

   c. Implicit Euler
      To improve stability, the Implicit Euler method evaluates the gradient at the future (unknown) time step. This requires solving a linear system at each iteration involving the matrix inversion of $(I - hA)$:

$$X_{i+1} = X_i + hF(X_{i+1})$$
$$X_{i+1} = X_i + h(AX_{i+1})$$
$$(I - hA)X_{i+1} = X_i$$
$$X_{i+1} = (I - hA)^{-1}X_i$$

      This method is generally more robust for stiff equations but computationally more intensive per step.

   d. Runge-Kutta 4 (RK4)

The fourth-order Runge-Kutta method provides a high-accuracy approximation by taking a weighted average of four slopes calculated within the time step. This method achieves the accuracy of a Taylor series expansion up to the fourth term without calculating higher derivatives:

$$X_{i+1} = X_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)h$$

Where

$$k_1 = F(X_i)$$
$$k_2 = F\left(X_i + \frac{1}{2}k_1 h\right)$$
$$k_3 = F\left(X_i + \frac{1}{2}k_2 h\right)$$
$$k_4 = F(X_i + k_3 h)$$

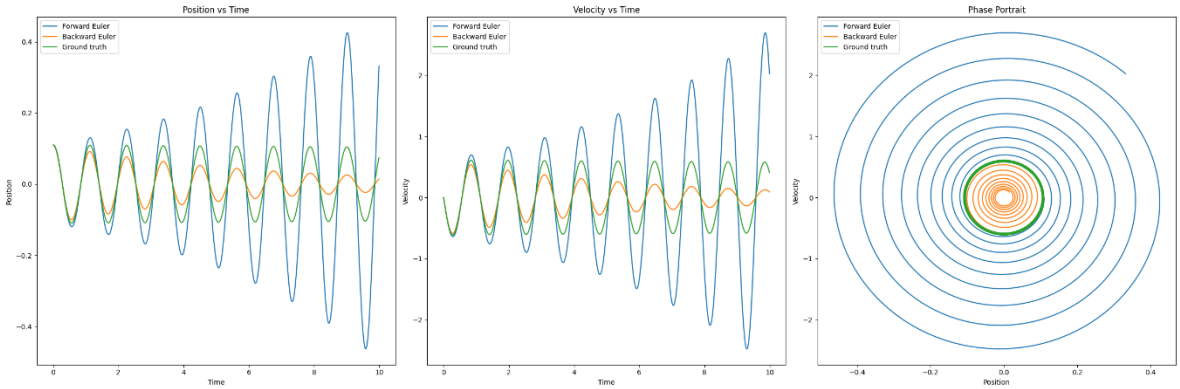
and $F(\mathbf{X}) = A\mathbf{X}$.

5. Results:

The simulation was conducted over a time interval from t=0 to t=50 seconds, divided into 5000 discrete time steps.

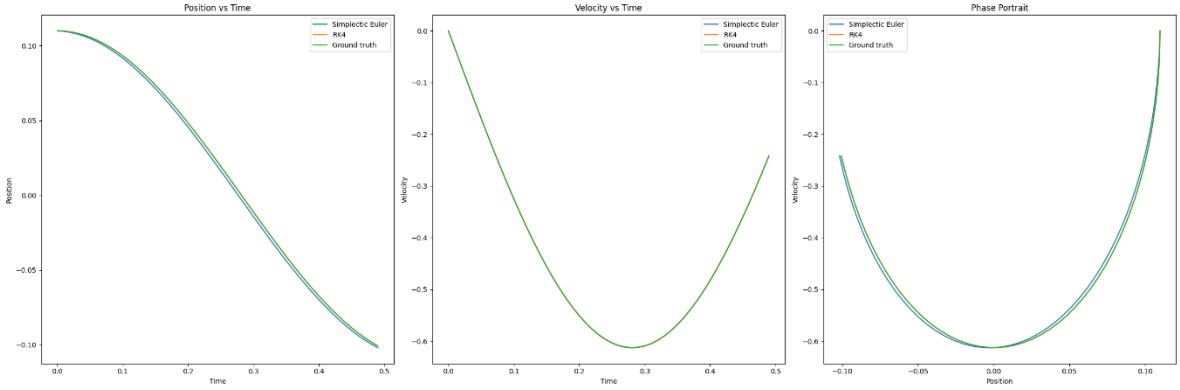
Sistema 1:

Revisamos los resultados para el metodo Bacward y Forward Euler:

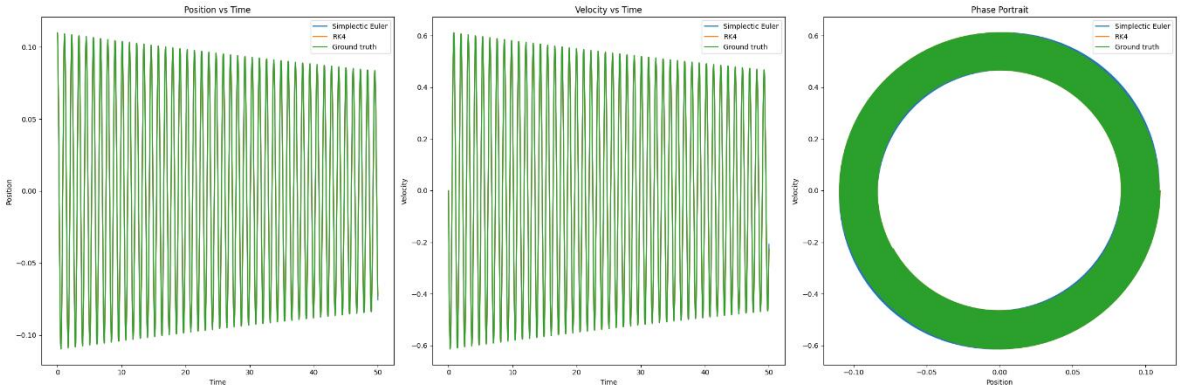


Podemos observer que el metodo explicito va aumentando el error, mientras que el implicito va decreciendo, en ambos casos se alejan de la función original

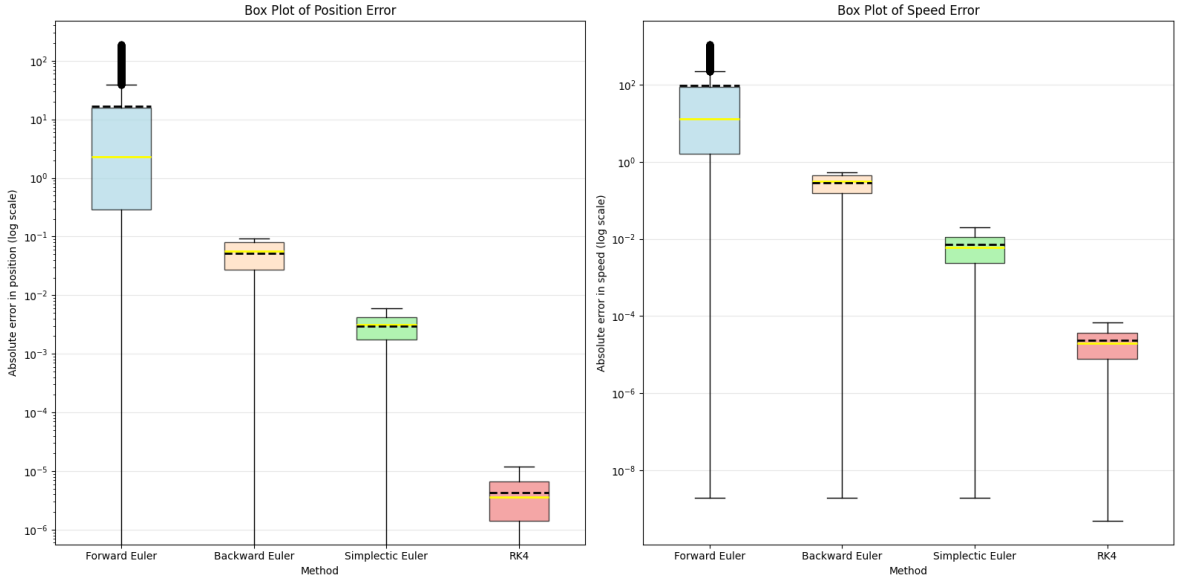
Revisamos el metodo simplectico y runge kutta:



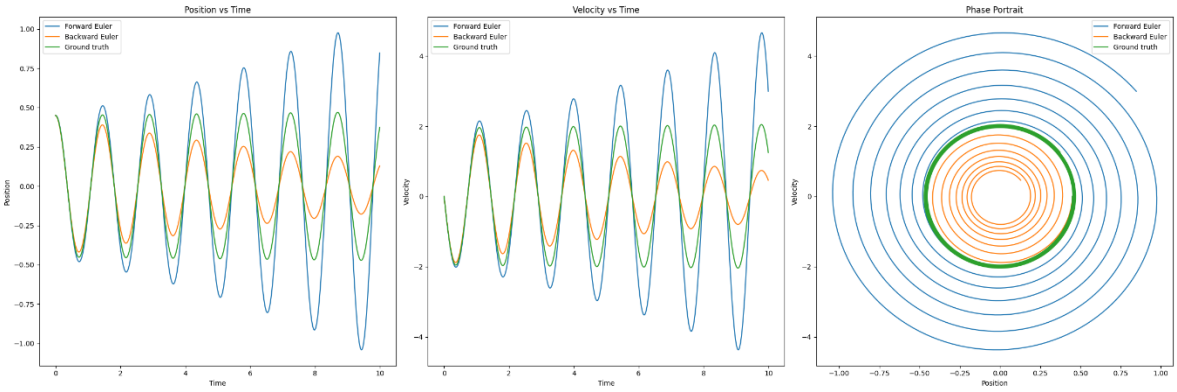
Se puede observar que el metodo simplectico presenta un comportamiento mas cercano a la función original, mientras que el metodo rk4 esta directamente sobre la original, aunque se puede apreciar que el metodo simplectico genera buenos resultados sin la cantidad de cálculos del metodo rk4



Estos resultados se mantienen en el tiempo
Si comparamos el valor absoluto del error en el tiempo, obtenemos la siguiente distribución, que permite observar que el mejor metodo es el rk4 seguido de simplectic euler

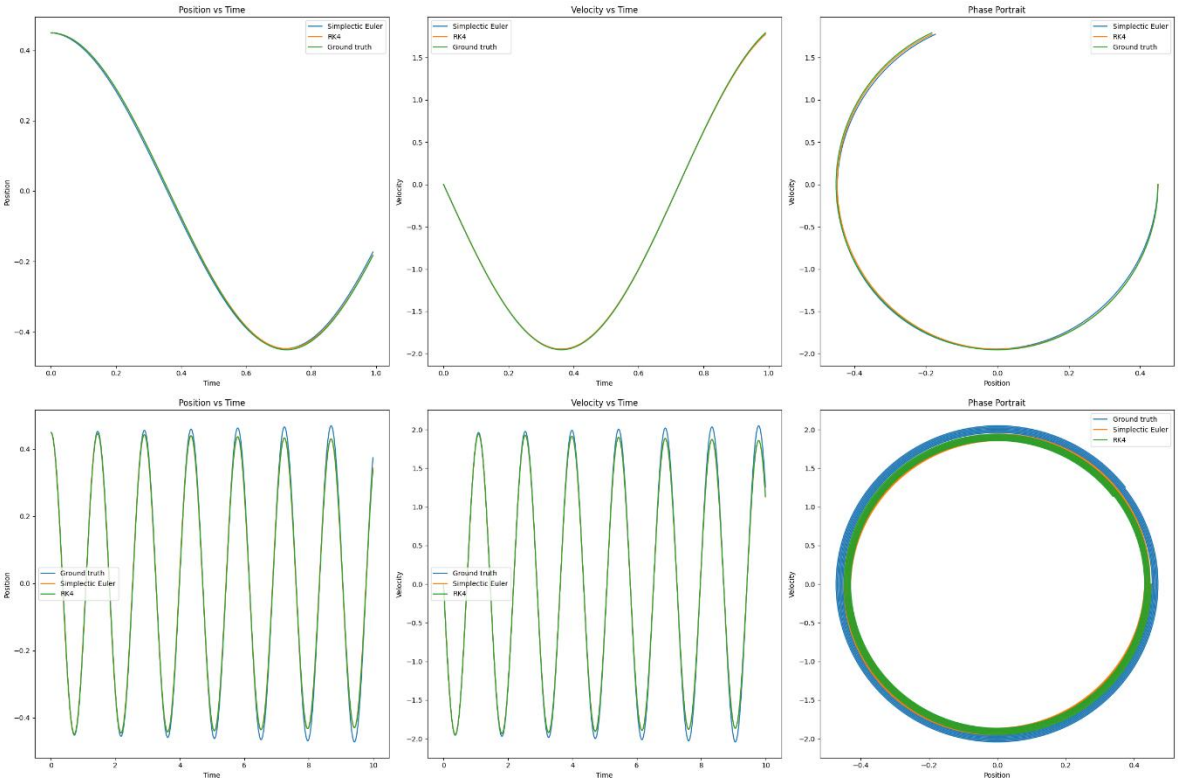


Sistema 2:
Revisamos los resultados para el metodo Bacward y Forward Euler:

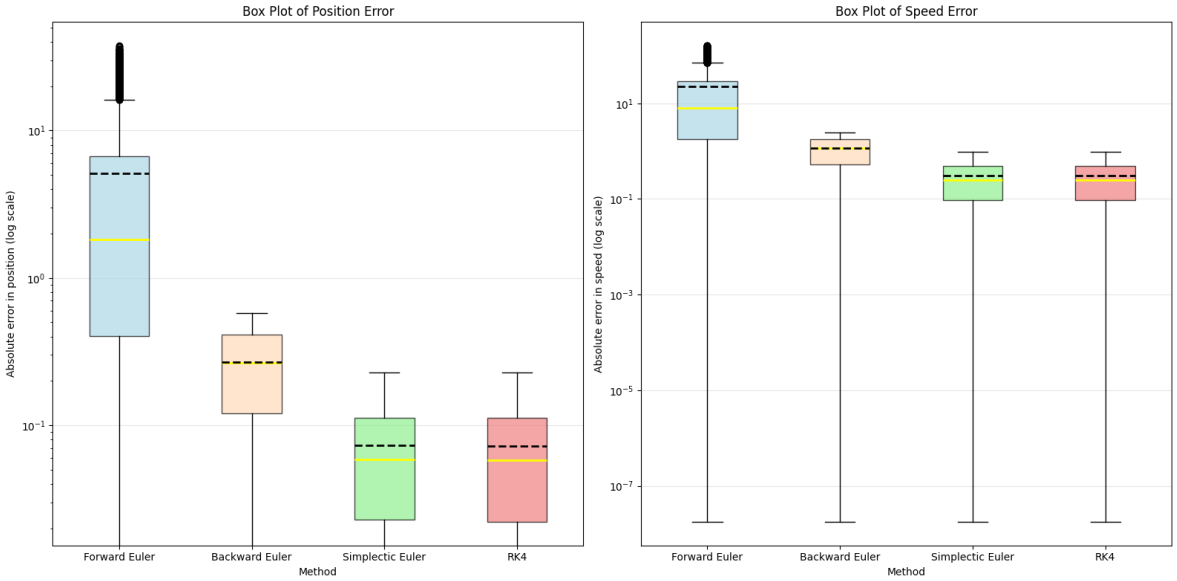


Al igual que en el caso anterior estos métodos rápidamente divergen de la solución real.

Revisamos el metodo simplectico y runge kutta:



La distribución del error entre los métodos:



Como se observo en los diagramas previos, los errores mas bajos son del metodo rk4 y simplectic euler

6. Conclusions

- **Accuracy Comparison:** All implemented numerical methods successfully approximated the general behavior of the analytical solution; however, the precision varied significantly. The Runge-Kutta 4 (RK4) method proved to be the most accurate, maintaining errors on the order of 10^{-4}. In contrast, both Euler methods exhibited errors on the order of 10^{-1}, is three orders of magnitude higher.
- The **Explicit Euler** method showed a tendency to accumulate error by overestimating the solution energy, leading to a larger spiral than the true trajectory.
- The **Implicit Euler** method demonstrated better numerical stability properties but tended to dampen the system, underestimating the true solution amplitude.
- The results highlight the trade-off between computational simplicity and numerical accuracy, with RK4 being the most reliable on the group.

7. Code

```
def analytical_solution(t):
    A = 0.1956
    B = -1.4823e-3
    omega = 1.2106
    k = 9.174e-3

    exp_kt = np.exp(k * t)
    cos_omega_t = np.cos(omega * t)
    sin_omega_t = np.sin(omega * t)
```

```
    x = exp_kt * (A * cos_omega_t + B * sin_omega_t) - A
    dx = (k * exp_kt * (A * cos_omega_t + B * sin_omega_t) +
          exp_kt * (-A * omega * sin_omega_t + B * omega *
cos_omega_t))

    return np.array([x, dx])
```

```
def backward_euler_2(fun, x0, Tf, h):
    x_hist = np.zeros((len(x0), len(t)))
    x_hist[:, 0] = x0
    a = -4.36
    b = 0.08
    c = -6.39
    d = 1.25
    ca = c/a
    ba = b/a
    da = d/a
    A = np.array([[0, 1],
                  [-ca, -ba]])
    B = np.array([0, da])
    K = np.eye(2)-A*h
    Kinv = np.linalg.inv(K)

    for k in range(len(t) - 1):
        x_hist[:, k + 1] = Kinv@(x_hist[:, k]+h*B)

    return x_hist
```